Anusuya – Emergency Request & Notification Module

Oct 10, 2026 · @Naruti

Module 4 of 4, owned by Anusuya: emergency requests, pharmacy acceptance, delivery or self-pickup, order tracking and in-app notifications for the whole application.

1. Module description and objectives

Anusuya's module turns a chosen pharmacy and medicine into a tracked emergency request that the pharmacy accepts or rejects, that either a delivery partner or the requester completes (self-pickup), and that keeps stock correct at every step, including cancellations. It also owns delivery partner availability and assignment, and the in-app notifications for the whole application.

Technology: HTML, CSS, JavaScript, Bootstrap 5 (frontend); Java 17 with Spring Boot 3, Spring Data JPA, Maven (backend); MySQL 8 with Flyway migrations (database); in-app notifications only.

Objectives

- Create emergency medicine requests with priority and prescription handling.
- Let pharmacies accept or reject, and expire requests nobody answers.
- Assign delivery partners, or fall back to self-pickup when none is available.
- Track every order status with a timeline and history.
- Notify every affected user in the application.
- Cancel safely and restore inventory without ever changing stock twice.

2. Features

- Request creation by patients and hospitals: 1 to 5 items from one pharmacy, priority `CRITICAL` or `URGENT`, delivery or self-pickup, patient details, doctor details for hospitals.
- Prescription upload (JPG, PNG, PDF, max 2 MB) and pharmacy verification for prescription-only items.
- Pharmacy accept or reject with reason; automatic expiry after 15 minutes.
- Delivery partner availability, location sharing, offers (nearest first, 2-minute timeout, up to 3 attempts) and assignment.

- Self-pickup with a 120-minute stock hold and a Navigate button (Deepika's navigation endpoint).
- Status tracking with timeline, 4-digit handover code for deliveries, and full request history.
- In-app notifications with unread badge in the navbar.
- Cancellation with stock release or restoration. Payment is cash at pharmacy or on delivery; there is no online payment.

3. Workflow

1. From a search result the requester opens `request/new.html`, prefilled with pharmacy, medicine, quantity and fulfilment type.
2. On submit the backend validates everything and calls Leelavathy's `reserve` for each item in one transaction. Status becomes `PENDING` and the pharmacy is notified (red badge for `CRITICAL`).
3. The pharmacy checks the prescription (Verify or Reject), then accepts or rejects. Reject needs a reason and releases the stock.
4. After accept: pickup requests become `READY_FOR_PICKUP`; delivery requests become `FINDING_DELIVERY` and the nearest available partner receives an offer.
5. The partner accepts (`DELIVERY_ASSIGNED`), collects the order (`OUT_FOR_DELIVERY`, stock committed) and delivers it with the requester's 4-digit code (`DELIVERED`).
6. If three offers fail, the requester and pharmacy are told, and the requester presses Switch to self-pickup (or the pharmacy retries delivery).
7. Pickup: the pharmacy presses Hand over (`PICKED_UP`, stock committed).
8. Every change writes `request_status_history` and sends notifications.

Status transitions (enforced by `RequestStateMachine`)

From	To	Actor	Stock effect
(new)	`PENDING`	Requester	`reserve`
`PENDING`	`ACCEPTED`, then `READY_FOR_PICKUP` or `FINDING_DELIVERY`	Pharmacy	none
`PENDING`	`REJECTED`	Pharmacy	`release`
`PENDING`	`EXPIRED` after 15 min	System	`release`

From	To	Actor	Stock effect
`FINDING_DELIVERY`	`DELIVERY_ASSIGNED`	Partner	none
`FINDING_DELIVERY`	`READY_FOR_PICKUP`	Requester	none
`DELIVERY_ASSIGNED`	`OUT_FOR_DELIVERY`	Partner	`commit`
`OUT_FOR_DELIVERY`	`DELIVERED`	Partner with code	none
`READY_FOR_PICKUP`	`PICKED_UP`	Pharmacy	`commit`
`READY_FOR_PICKUP`	`EXPIRED` after 120 min	System	`release`
`PENDING`, `ACCEPTED`, `FINDING_DELIVERY`, `DELIVERY_ASSIGNED`, `READY_FOR_PICKUP`	`CANCELLED`	Requester, pharmacy or admin	`release`
`OUT_FOR_DELIVERY`	`CANCELLED`	Pharmacy or admin only	`restore`
`DELIVERED`, `PICKED_UP`, `REJECTED`, `EXPIRED`, `CANCELLED`	none	none	Terminal

4. Frontend requirements

Use HTML, CSS, JavaScript and Bootstrap 5.3. Reuse Divya's `theme.css`, `api.js`, `ui.js` and `auth-guard.js`: Navy `#102A43` primary, Teal `#0D9488` secondary, Red `#DC2626` for emergency alerts and the Submit emergency request button, backgrounds `#F1F5F9` and white, font Inter. Add only `request.css`, `request.js`, `track.js`, `delivery.js` and `notifications.js`. Every page needs a loading spinner, empty state, inline validation and toasts.

Page	Purpose and active controls
`request/new.html`	Four steps. 1 Items: prefilled, quantity stepper, Add item (max 5, same pharmacy). 2 Fulfilment: Delivery or Self-pickup; Delivery disabled with an explanation when unavailable; delivery address, Use my location, fee shown. 3 Details: patient name, phone, priority (Critical or Urgent), notes; doctor name and registration number for hospitals; prescription upload with preview when needed. 4 Review: totals, a notice that the app gives no medical advice, and a red Submit emergency request button (`POST /requests`). Reads `pharmacyId` , `medicineId` , `quantity` , `fulfilment` from the URL.
`request/my-requests.html`	Tabs Active, Completed, Cancelled; table with reference, pharmacy, items, status badge, total, time; View; Cancel (modal asks reason, `POST /requests/{id}/cancel`)
`request/track.html?id=`	Reference and status badge; vertical timeline; items and totals; prescription status; pharmacy card with Call, WhatsApp (`GET /contact/{id}/links`) and Navigate (`GET /navigation/{id}`) from Deepika's module; partner card; red handover-code card for deliveries; Cancel; Switch to self-pickup; polls every 10 seconds
`pharmacy/requests.html`	Tabs New, In progress, Completed. Card per request with priority badge, items with current stock, prescription viewer (Verify, Reject), expiry countdown. Buttons: Accept, Reject (reason modal), Retry delivery, Hand over, Cancel
`delivery/jobs.html`	Availability switch, Share my location, offers with countdown (Accept, Reject), active job with Navigate links, Picked up, Delivered with code input
`delivery/history.html`	Table of finished jobs with paging

Page	Purpose and active controls
`notifications.html`	List with unread highlight, icon and colour by type (red for emergency types), Mark as read, Mark all read; click opens the related request. `notifications.js` updates the navbar bell every 15 seconds.
`admin/requests.html`	Counters by status, filters (status, pharmacy, date), table with View and Cancel

5. Spring Boot backend requirements

- Request creation, validation, price snapshots and reference code `EMR-YYYYMMDD-NNNN`.
- `RequestStateMachine` is the only place that changes status; every change writes `request_status_history` and sends notifications; an illegal move returns `INVALID_STATE_TRANSITION`.
- Stock calls to Leelavathy's `InventoryService` (`reserve` , `commit` , `release` , `restore`) carry the request reference code and share one `@Transactional` boundary with the status change. `emergency_requests.stock_committed` guards `restore` .
- Prescription storage outside the web root (property `app.upload-dir`), random UUID file names, content type checked from file bytes, access checks on download.
- Delivery assignment: partners with `is_available = true` , verified, active, no active job, last known location within 15 km of the pharmacy (no location ranks last), nearest first; offer expires after 2 minutes; at most 3 attempts.
- Scheduled jobs every minute: expire `PENDING` after 15 minutes (property `app.request.pending-expiry-minutes`), expire `READY_FOR_PICKUP` after 120 minutes (`app.request.pickup-hold-minutes`), expire offers and offer to the next partner.
- Implement `NotificationPublisher` and `DeliveryAvailabilityQuery` (`AVAILABLE` if delivery is enabled and a partner is free, `BUSY` if enabled but all busy, otherwise `UNAVAILABLE`) for the whole application.
- Summary endpoint for dashboards; `RequestDemoDataLoader` (order 400) creating three sample requests (`PENDING` , `READY_FOR_PICKUP` , `DELIVERED`) through the services.
- Flyway versions `V400` to `V499` .

Notification events

Event	Recipient	Type
Request created	Pharmacy	`REQUEST_CREATED`
Accepted	Requester	`REQUEST_ACCEPTED`
Rejected or expired	Requester	`REQUEST_REJECTED`
Delivery offer	Partner	`DELIVERY_OFFER`
Assigned, picked up, delivered, ready for pickup, cancelled, no partner	Requester and pharmacy as relevant	`STATUS_UPDATE`
Pharmacy approved or rejected (published by Divya's module)	Pharmacy	`PHARMACY_APPROVED`, `PHARMACY_REJECTED`
Stock low (published by Leelavathy's module)	Pharmacy	`LOW_STOCK`

```
com.emergencymed.request
  controller/ EmergencyRequestController, PrescriptionController,
DeliveryController,
              NotificationController, AdminRequestController
  service/    RequestService, RequestStateMachine, PrescriptionService,
              DeliveryAssignmentService, RequestSummaryService,
ReferenceCodeGenerator
  service/impl/ NotificationService (implements NotificationPublisher),
              DeliveryAvailabilityService (implements
DeliveryAvailabilityQuery)
  storage/    FileStorageService
  repository/ EmergencyRequestRepository, RequestItemRepository,
PrescriptionRepository,
              RequestStatusHistoryRepository, DeliveryAvailabilityRepository,
              DeliveryAssignmentRepository, NotificationRepository
  entity/     EmergencyRequest, RequestItem, Prescription,
RequestStatusHistory,
              DeliveryAvailability, DeliveryAssignment, Notification
  dto/        CreateRequestDto, RequestDetailResponse, TimelineItem,
OfferResponse, NotificationResponse
  job/        RequestExpiryJob, AssignmentExpiryJob
  demo/       RequestDemoDataLoader
src/main/resources/db/migration  V400__m4_requests.sql,
V410__m4_delivery.sql, V420__m4_notifications.sql
src/main/resources/static/request/    new.html, my-requests.html, track.html
src/main/resources/static/pharmacy/   requests.html
```

```
src/main/resources/static/delivery/   jobs.html, history.html
src/main/resources/static/admin/      requests.html
src/main/resources/static/            notifications.html
src/main/resources/static/assets/     css/request.css, js/request.js,
js/track.js, js/delivery.js, js/notifications.js
```

6. MySQL database tables (owned by Anusuya)

Seven tables, Flyway files `V400__m4_requests.sql`, `V410__m4_delivery.sql`, `V420__m4_notifications.sql`, created after all other modules' migrations.

`emergency_requests`: `id` PK; `reference_code` VARCHAR(20) UNIQUE; `requester_user_id` FK to `users.id`; `requester_type` VARCHAR(10); `pharmacy_id` FK to `pharmacies.id`; `fulfilment_type` VARCHAR(10); `priority` VARCHAR(10); `status` VARCHAR(20); `patient_name` VARCHAR(100); `patient_phone` VARCHAR(15); `doctor_name` VARCHAR(100) NULL; `doctor_reg_no` VARCHAR(40) NULL; `delivery_address` VARCHAR(255) NULL; `delivery_lat`, `delivery_lng` DECIMAL(9,6) NULL; `delivery_partner_user_id` FK to `users.id` NULL; `items_total`, `delivery_fee`, `grand_total` DECIMAL(10,2); `payment_mode` VARCHAR(20) (`PAY_AT_PHARMACY` or `CASH_ON_DELIVERY`); `prescription_status` VARCHAR(15); `handover_code` CHAR(4) NULL; `delivery_attempts` INT default 0; `stock_committed` TINYINT(1) default 0; `notes` VARCHAR(500); `rejection_reason`, `cancel_reason` VARCHAR(255); `cancelled_by_role` VARCHAR(10); `expires_at` DATETIME; `accepted_at`, `completed_at` DATETIME NULL; `created_at`, `updated_at`. Indexes: (`requester_user_id`, `status`), (`pharmacy_id`, `status`), (`status`, `expires_at`).

`request_items`: `id` PK; `request_id` FK to `emergency_requests.id`; `medicine_id` FK to `medicines.id`; `inventory_id` FK to `pharmacy_inventory.id`; `medicine_name` VARCHAR(160) (snapshot); `quantity` INT; `unit_price` DECIMAL(10,2) (snapshot); `line_total` DECIMAL(10,2); `requires_prescription` TINYINT(1) (snapshot).

`prescriptions`: `id` PK; `request_id` FK to `emergency_requests.id` NULL until attached; `uploaded_by_user_id` FK to `users.id`; `file_name` VARCHAR(120) (generated UUID name); `original_name` VARCHAR(160); `content_type` VARCHAR(40); `file_size` INT; `doctor_name` VARCHAR(100) NULL; `verification_status` VARCHAR(15) default `PENDING`; `verified_by_user_id` FK to `users.id` NULL; `verified_at` DATETIME NULL; `remarks` VARCHAR(255); `created_at`.

`request_status_history`: `id` PK; `request_id` FK; `from_status`, `to_status` VARCHAR(20); `changed_by_user_id` FK to `users.id` NULL (NULL for system); `note` VARCHAR(255); `created_at`.

`delivery_availability`: `id` PK; `partner_user_id` FK to `users.id` UNIQUE; `is_available` TINYINT(1); `current_lat`, `current_lng` DECIMAL(9,6) NULL; `last_location_at` DATETIME NULL; `updated_at`.

`delivery_assignments`: `id` PK; `request_id` FK; `partner_user_id` FK to `users.id`; `status` VARCHAR(10) (`OFFERED`, `ACCEPTED`, `REJECTED`, `EXPIRED`); `attempt_no` INT; `offered_at`, `expires_at`, `responded_at` DATETIME; `reject_reason` VARCHAR(255) NULL. Index (`partner_user_id`, `status`).

`notifications`: `id` PK; `user_id` FK to `users.id`; `request_id` FK to `emergency_requests.id` NULL; `type` VARCHAR(25); `title` VARCHAR(120); `message` VARCHAR(500); `is_read` TINYINT(1) default 0; `created_at`; `read_at` NULL. Index (`user_id`, `is_read`, `created_at`).

Relationships: request 1 to many items, history rows and assignments; request 0 or 1 prescription; user 1 to many requests (as requester); pharmacy 1 to many requests; partner user 1 to 1 availability; user 1 to many notifications.

7. REST APIs

Base path `/api/v1`, shared response envelope.

Method	Path	Role	Purpose
POST	`/requests`	PATIENT, HOSPITAL	Create request
GET	`/requests/mine`	PATIENT, HOSPITAL	Own requests; `status`, paging
GET	`/requests/{id}`	Requester, pharmacy, assigned partner, ADMIN	Full detail with items, history, prescription, partner
GET	`/requests/summary`	Any	Role-based counts for dashboards
POST	`/requests/{id}/cancel`	Requester, PHARMACY, ADMIN	Body `{reason}`
POST	`/requests/{id}/switch-to-pickup`	Requester	Only in `FINDING_DELIVERY`
GET	`/requests/pharmacy`	PHARMACY	Incoming requests; `status`, paging
POST	`/requests/{id}/accept`	PHARMACY	Accept

Method	Path	Role	Purpose
POST	`/requests/{id}/reject`	PHARMACY	Body `{reason}`
POST	`/requests/{id}/retry-delivery`	PHARMACY	Offer to partners again
POST	`/requests/{id}/handover`	PHARMACY	Pickup completed
POST	`/prescriptions/upload`	PATIENT, HOSPITAL	Multipart file plus optional `doctorName`; returns `prescriptionId`
GET	`/prescriptions/{id}/file`	Owner, pharmacy of request, ADMIN	Stream image or PDF
PATCH	`/prescriptions/{id}/verify`	PHARMACY	Body `{status, remarks}`
GET	`/delivery/me/availability`	DELIVERY	Current status
PUT	`/delivery/me/availability`	DELIVERY	Body `{available, lat?, lng?}`
PUT	`/delivery/me/location`	DELIVERY	Body `{lat, lng}`
GET	`/delivery/offers`	DELIVERY	Open offers
POST	`/delivery/offers/{assignmentId}/accept`	DELIVERY	Accept
POST	`/delivery/offers/{assignmentId}/reject`	DELIVERY	Reject with optional reason
GET	`/delivery/jobs`	DELIVERY	`type=active` or `history`, paging
POST	`/delivery/jobs/{requestId}/picked-up`	DELIVERY	Commit stock, `OUT_FOR_DELIVERY`
POST	`/delivery/jobs/{requestId}/delivered`	DELIVERY	Body `{handoverCode}`
GET	`/notifications`	Any	Own notifications; `unreadOnly`, paging
GET	`/notifications/unread-count`	Any	`{count}`
PATCH	`/notifications/{id}/read`	Any	Mark one read
PATCH	`/notifications/read-all`	Any	Mark all read
GET	`/admin/requests`	ADMIN	Filters `status`, `pharmacyId`, `from`, `to`

Create request body

```
{
  "pharmacyId": 1,
  "fulfilmentType": "DELIVERY",
  "priority": "CRITICAL",
  "items": [ { "medicineId": 4, "quantity": 2 } ],
  "patientName": "Asha Raman",
  "patientPhone": "9000000011",
  "deliveryAddress": "Flat 4B, Sample Apartments, Adyar, Chennai",
  "deliveryLat": 13.0050,
  "deliveryLng": 80.2550,
  "prescriptionId": 12,
  "notes": "Patient is in respiratory distress"
}
```

8. Integration requirements

What Anusuya provides and consumes

- Provides to everyone: `NotificationPublisher` (used by Divya for approvals, Leelavathy for low stock, and by her own request flow), `DeliveryAvailabilityQuery` (used by Deepika), `GET /requests/summary` for dashboards, and the notification bell script.
- Consumes from Divya: `CurrentUserProvider`, `PharmacyLookup`, `UserLookup.activeDeliveryPartnerIds`, `GeoUtils`, `MapLinks`, theme and `ui.js`.
- Consumes from Leelavathy: `InventoryService` (`reserve`, `commit`, `release`, `restore`), `MedicineLookup`, `PharmacySettingsLookup` (delivery enabled, radius, fee).
- Consumes from Deepika through the browser only: `GET /contact/{id}/links`, `GET /navigation/{id}` and the page `/pharmacy-details.html`. Her form is opened with `?pharmacyId=&medicineId=&quantity=&fulfilment=`.
- Rules for stock calls: every call carries the request `reference_code`; Leelavathy's methods are idempotent per inventory row, reference code and transaction type; reservation, status change and history are in one `@Transactional` boundary.

Shared conventions (identical in all four module documents)

- Base path `/api/v1`. Every response is `{success, message, data, errors[{field,message}], timestamp}`. Lists use `data = {content, page, size, totalElements, totalPages}` with params `page` (from 0), `size` (default 10, maximum 50) and `sort`.
- Roles: `PATIENT`, `HOSPITAL`, `PHARMACY`, `DELIVERY`, `ADMIN`. The user and pharmacy ID always come from the session through `CurrentUserProvider`, never from the request body.

- Naming: tables plural `snake_case`, entities singular `PascalCase`, JSON `camelCase`, enums `UPPER_SNAKE` stored as strings, `BIGINT` IDs, money `DECIMAL(10,2)` in INR, coordinates `DECIMAL(9,6)`, timestamps in UTC.
- Login is a server-side session; every `POST`, `PUT`, `PATCH`, `DELETE` carries the CSRF header `X-XSRF-TOKEN` (added by `api.js`).
- Stock: `availableQuantity = quantity - reservedQuantity`. Inventory rows are `SAMPLE` or `LIVE`; sample data is never presented as real-time stock.
- Prescription-only medicines need a prescription verified by the pharmacy before a request is accepted.

HTTP	`errorCode`	When
400	`VALIDATION_ERROR`	Bean validation or bad parameter
401	`UNAUTHORIZED`	Not logged in or session expired
403	`FORBIDDEN`	Wrong role or another user's data
403	`PHARMACY_NOT_APPROVED`	Pharmacy action before approval
404	`NOT_FOUND`	Missing record
409	`DUPLICATE_RESOURCE`	Email, licence number or medicine already exists
409	`INSUFFICIENT_STOCK`	Reservation larger than available stock
409	`INVALID_STATE_TRANSITION`	Status change not allowed
409	`NO_DELIVERY_PARTNER`	Delivery not possible; offer self-pickup
422	`PRESCRIPTION_REQUIRED`	Prescription-only item without a verified prescription
500	`INTERNAL_ERROR`	Unexpected; generic message only

Shared database design (one MySQL database `emergency_med_db`, 19 tables)

Owner	Module	Tables	Flyway range
Divya	User & Authentication	`users`, `patient_profiles`, `hospital_profiles`, `pharmacies`, `delivery_partner_profiles`	`V100` to `V199`

Owner	Module	Tables	Flyway range
Leelavathy	Medicine & Inventory	`medicine_categories`, `medicines`, `pharmacy_settings`, `pharmacy_inventory`, `inventory_transactions`	`V200` to `V299`
Deepika	Medicine Search & Availability	`search_history`, `contact_logs`	`V300` to `V399`
Anusuya	Emergency Request & Notification	`emergency_requests`, `request_items`, `prescriptions`, `request_status_history`, `delivery_availability`, `delivery_assignments`, `notifications`	`V400` to `V499`

Only the owner reads or writes a table directly. Anusuya's tables have foreign keys to Divya's `users` and `pharmacies` and to Leelavathy's `medicines` and `pharmacy_inventory`, so her migrations run last. Stock numbers are changed only through `InventoryService`, never by SQL from this module. No `ON DELETE CASCADE` across modules.

Interface (`com.emergencymed.common.contract`)	Implemented by	Used by	Methods
`CurrentUserProvider`	Divya	all	`Long userId()`, `Role role()`, `Optional<Long> pharmacyId()`, `boolean isAuthenticated()`
`PharmacyLookup`	Divya	Leelavathy, Deepika, Anusuya	`findApproved(id)`, `findApprovedWithin(lat, lng, radiusKm)`, `pharmacyIdForUser(userId)`
`UserLookup`	Divya	Anusuya	`findById(userId)`, `activeDeliveryPartnerIds()`
`MedicineLookup`	Leelavathy	Deepika, Anusuya	`findById(id)`, `searchByName(q, limit)`
`InventoryQueryService`	Leelavathy	Deepika	`findOffers(medicineId, qty, pharmacyIds)`, `findAlternatives(medicineId, pharmacyIds)`

Interface (`com.emergencymed.common.contract`)	Implemented by	Used by	Methods
`InventoryService`	Leelavathy	Anusuya	`Long reserve(pharmacyId, medicineId, qty, refCode)`, `commit`, `release`, `restore` (inventoryId, qty, refCode)
`PharmacySettingsLookup`	Leelavathy	Deepika, Anusuya	`find(pharmacyId)`
`DeliveryAvailabilityQuery`	Anusuya	Deepika	`statusFor(pharmacyId)`: `AVAILABLE`, `BUSY`, `UNAVAILABLE`
`NotificationPublisher`	Anusuya	Divya, Leelavathy, Anusuya	`publish(userId, type, title, message, requestId)`

Every interface Anusuya consumes needs a `@ConditionalOnMissingBean` fallback so her module runs alone before integration.

9. Validation, exception handling and security

Input	Rule
Items	1 to 5 distinct medicines; quantity 1 to 100 each; all from one approved, active pharmacy
Stock	Reserve must succeed for every item, otherwise 409 `INSUFFICIENT_STOCK` and nothing is kept
Prescription	Required if any item needs one, else 422 `PRESCRIPTION_REQUIRED`; JPG, PNG or PDF up to 2 MB; type checked from file bytes; stored under a random UUID name
Patient phone	`^[6-9][0-9]{9}$`
Delivery	Pharmacy `delivery_enabled`, address 10 to 255 characters, coordinates within the pharmacy delivery radius, otherwise 409 `NO_DELIVERY_PARTNER` with a message to choose self-pickup
Reasons	Reject at least 10 characters; cancel at least 5
Handover code	Exactly 4 digits; 3 wrong attempts return 403 and notify the pharmacy

Input	Rule
Duplicate submit	Same requester, pharmacy and items within 2 minutes returns 409 `DUPLICATE_RESOURCE`

- All status changes go through `RequestStateMachine`; anything else returns 409 `INVALID_STATE_TRANSITION`.
- Ownership: a requester sees only their own requests; a pharmacy only its own; a partner only offers and jobs assigned to them; others get 403 or 404.
- Prescription files are never served from a public path; downloads need the access check.
- An unapproved pharmacy cannot accept requests.
- `commit` failure rolls back the status change.
- The application gives no medical advice; the verifying pharmacist is responsible for dispensing decisions. Show this notice on `request/new.html`.

10. Testing scenarios and expected output

Scenario	Steps	Expected
Happy path, pickup	Create `PICKUP`, pharmacy accepts, hands over	`PENDING`, `READY_FOR_PICKUP`, `PICKED_UP`; stock lowered once; history rows and notifications at each step
Happy path, delivery	Create `DELIVERY`, accept, rider1 accepts, picks up, delivers with code	Ends `DELIVERED`; stock committed at pick-up; code required
Rx missing	Request Adrenaline without a prescription	422 `PRESCRIPTION_REQUIRED`; no reservation
Rx not verified	Pharmacy presses Accept while the prescription is `PENDING`	409 with a message to verify first
Over-request	Request more than available	409 `INSUFFICIENT_STOCK`; stock unchanged
Two buyers, one unit	Two users request the last unit at the same time	One `PENDING`; the other 409
Reject	Pharmacy rejects with a reason	`REJECTED`; reserved quantity back to 0; requester notified with the reason

Scenario	Steps	Expected
Expiry	Leave a request untouched (use a 1-minute test setting)	`EXPIRED`; stock released
No partner	Both riders offline or reject	After 3 attempts the requester sees Switch to self-pickup; pressing it gives `READY_FOR_PICKUP` and removes the fee
Delivery disabled	Choose `DELIVERY` for CityMeds (delivery off)	Radio disabled; API returns 409 `NO_DELIVERY_PARTNER`
Cancel before dispatch	Requester cancels in `DELIVERY_ASSIGNED`	`CANCELLED`; stock released; partner free again
Cancel after dispatch	Requester tries to cancel in `OUT_FOR_DELIVERY`	409 `INVALID_STATE_TRANSITION`; pharmacy cancel succeeds and `restore` returns stock
Double click	Press Accept twice quickly	One transition; second call 409; stock unchanged
Wrong handover code	Partner enters 0000	403; status unchanged
Access control	Patient A opens a request of patient B	404 `NOT_FOUND`
Notifications	Create a request, open the pharmacy bell	Unread count 1; click opens the request; count drops to 0